

TECHNICAL REPORT

EEG Cognitive Stress Classification

Methodology, design decisions, and findings on the SAM 40 dataset with subject-grouped cross-validation

PROJECT

End-to-end ML pipeline for binary cognitive stress classification from EEG signals

DATASET

SAM 40 (40 subjects, 32-channel EEG, 128 Hz, 4 tasks)

EVALUATION

Subject-grouped held-out and 40-fold leave-one-subject-out cross-validation

FINAL RESULT

0.587 (classical ML, LOSO) / 0.552 (deep learning, LOSO)

EEG Cognitive Stress Classification with Subject-Grouped Cross-Validation

A detailed account of the methodology, design decisions, diagnostics, and findings of the EEG stress classification project. Read this if you've read the README and want to understand the *why* behind each step, or if you're trying to reproduce or extend the work.

Table of contents

- Abstract
 - 1. Introduction
 - 2. Dataset
 - 3. Phase 1 — Data Exploration
 - 4. Phase 2 — Preprocessing
 - 5. Phase 3 — Feature Extraction
 - 6. Phase 4 — Classical ML Classification
 - 7. Phase 4B — Improved Classification
 - 8. Phase 5 — Deep Learning
 - 9. Results and Discussion
 - 10. Limitations
 - 11. Engineering Decisions
 - 12. Future Work
 - 13. Conclusion
 - References
 - Appendix A — Hyperparameter Summary
 - Appendix B — Per-Phase File Outputs
-

Abstract

This project builds an end-to-end machine learning pipeline for binary cognitive stress classification from EEG signals, using the SAM 40 dataset (40 subjects, 32-channel EEG at 128 Hz). The pipeline progresses through six phases: data exploration, preprocessing with per-subject normalization, feature extraction (480 hand-crafted features), classical machine learning (SVM, Random Forest, KNN) with leave-one-subject-out cross-validation (LOSO-CV), an improved classical ML phase with feature selection and hyperparameter tuning, four deep learning architectures (Simple CNN, EEGNet, and regularization variants of each), and finally a 40-fold LOSO-CV evaluation of the winning deep learning model.

The central methodological commitment is subject-grouped evaluation throughout. The defining finding is that **traditional machine learning with hand-crafted features (Random Forest, 0.587 LOSO accuracy) outperformed every deep learning configuration attempted, including the standard EEGNet baseline (0.525) and a custom CNN (0.552)**. This result is consistent with recent literature on small-cohort EEG classification and supports the broader observation that the spectacular accuracies reported in many EEG papers reflect data-leakage in their cross-validation schemes rather than genuine cross-subject generalization. The most important deliverable is therefore not the headline accuracy but a methodologically rigorous baseline against which future work on this dataset can be honestly compared.

1. Introduction

1.1 Cognitive stress and EEG

Cognitive stress — the mental load induced by demanding tasks — has measurable correlates in

electroencephalography (EEG). The most established markers involve power redistributions across the standard frequency bands: a decrease in alpha-band power (8–13 Hz) and a relative increase in beta-band power (13–30 Hz) during cognitive load, often summarized as a rising Beta/Alpha ratio. Frontal and parietal regions tend to show the largest effects. These patterns are subject-variable but reasonably consistent in direction.

The clinical and human-factors applications of EEG-based stress detection are well-known — monitoring drivers, pilots, students, or surgical staff during high-load tasks; deploying biofeedback for stress management; building closed-loop systems in human-computer interaction. The technical question is whether a model can learn these patterns from a small multi-subject EEG dataset in a way that *generalizes to new individuals* — and that is harder than it sounds.

1.2 The cross-subject generalization problem

EEG signals vary enormously across individuals. Resting alpha amplitudes can differ by factors of 5–10×. Electrode impedance, skull thickness, scalp tissue, hair density, and individual cortical anatomy all shift the recorded signal in ways that have nothing to do with the task being classified. A classifier trained on Subject A’s data may pick up patterns specific to Subject A’s resting EEG and fail entirely on Subject B.

The standard remedy is **leave-one-subject-out cross-validation (LOSO-CV)**: hold out one subject entirely, train on the others, repeat for every subject, average the results. This forces the model to find patterns that are robust across people, not artifacts that happen to correlate with one person’s idiosyncrasies.

When this kind of evaluation is applied honestly, EEG classification accuracies on small datasets typically drop into the 55–70% range for binary tasks. When it is *not* applied — when segments from the same subject appear in both train and test (as in k-fold CV without grouping) — the same architectures can report 85–95%. The model is then essentially performing subject identification while pretending to do task classification.

1.3 Project scope and contribution

This project takes the SAM 40 dataset and builds the entire stack — preprocessing, classical ML, deep learning — with subject-grouped evaluation from Phase 4 onward. The deliverables are:

1. A working pipeline that takes raw `.mat` files and outputs LOSO-evaluated stress predictions.
2. A comparison of three classical ML algorithms and four deep learning configurations under identical evaluation conditions.
3. Honest reporting: numbers, distributions, failure cases, and analysis of why some models worked better than others.
4. A clean, reproducible codebase suitable for extension to other EEG datasets or tasks.

2. Dataset

2.1 SAM 40 overview

SAM 40 (Ghosh et al., 2022) is a publicly available EEG stress dataset hosted on Figshare. Key parameters:

Property	Value
Subjects	40 (14 female, 26 male, mean age 21.5)
EEG device	Emotiv EPOC Flex (gel kit)
Channels	32
Sampling rate	128 Hz
Tasks	4 (Relaxation, Stroop, Arithmetic, Mirror Image)
Trials per task per subject	3
Total recordings	$40 \times 4 \times 3 = \mathbf{480}$
Approximate trial duration	25 s (3200 samples)
Format	MATLAB <code>.mat</code> , data key <code>Clean_data</code>
Labels	Self-reported stress rating per trial (in <code>scales.xls</code>)

The dataset is distributed in two versions: raw and filtered. The filtered version has already undergone band-pass filtering and artifact removal (eye blinks, muscle activity) by the original authors. This project uses the filtered version throughout, since reproducing their cleaning pipeline was not the goal.

The filename convention in the actual dataset differs from the paper description: files are named `Task_sub_N_trialM.mat` (e.g., `Arithmetic_sub_1_trial1.mat`), and the EEG data is stored under the `Clean_data` key inside each `.mat` file — not `data` as the paper implies.

2.2 Stress labels

The `scales.xls` file contains self-reported stress ratings on a 1-10 scale for each trial. These were discretized into three categories:

Class	Rating range	Number of segments	% of total
Relaxed	0-3	1110	46.2%
Low Stress	4-6	940	39.2%
High Stress	7-10	350	14.6%
Total	—	2400	100%

Three observations:

- **The three-class distribution is heavily imbalanced.** High Stress is only 14.6% — too few examples for a small deep learning model to learn well, and a strong source of overfitting in classical ML.
- **The binary split (Relaxed vs. Stress) is well-balanced.** 1110 vs 1290 (46.2% / 53.8%). This was the primary task for Phase 5.
- **The labels are self-reported, hence noisy.** A subject's rating depends on their personality and frame of reference. Some "Stress" trials may have produced very mild physiological response, and vice versa.

2.3 Why these tasks

The three cognitive tasks were selected by the dataset authors to induce stress through different mechanisms: visual interference (Stroop), arithmetic load (Arithmetic), and spatial cognition (Mirror Image). Trying to classify *which* task is being performed is a separate question from classifying *whether stress is being induced*. This project focuses on the latter — binary stress vs. relaxation — because it generalizes better and matches the realistic application of detecting stress regardless of its source.

3. Phase 1 — Data Exploration

3.1 Goals

Before any modelling, the dataset had to be verified for shape, integrity, and label consistency. Specifically:

- Could all 480 files be loaded without errors?
- Were the shapes consistent?
- Did the stress ratings in `scales.xls` align with the file naming?
- What did the raw signals actually look like in each condition?

3.2 Findings

- **All 480 files loaded successfully.** Shape per file: 32 channels × 3200 samples (25 seconds at 128 Hz).
- **Filename-to-label mapping required care.** The paper described a different naming scheme than the actual files, and `scales.xls` used 1-indexed subjects and tasks while the files used various conventions. Mapping logic was written defensively, with assertions verifying that every file matched a row in `scales.xls`.
- **Raw signals look reasonable.** Amplitudes typically in $\pm 60 \mu\text{V}$, with the larger transients (eye blinks, motion) already removed by the authors' artifact processing. Standard deviation per channel was $\sim 7.2 \mu\text{V}$ — consistent with filtered EEG.
- **The label distribution above** was confirmed via Phase 1's histogram analysis.

3.3 Phase 1 outputs

Saved to `Results/phase1/` :

- `01_raw_eeg_all_tasks.png` — example signals from each of the 4 tasks
 - `02_zoomed_relax_vs_arithmetic.png` — side-by-side comparison highlighting amplitude differences
 - `03_stress_label_distribution.png` — histogram of self-reported ratings
 - `04_channel_layout.png` — 10–20 system electrode positions
 - `05_data_quality_dashboard.png` — channel-level summary statistics
 - `stress_labels.csv` — clean label table for downstream use
-

4. Phase 2 — Preprocessing

4.1 Pipeline

The preprocessing chain consisted of three steps:

1. **Power Spectral Density (PSD) analysis** for quality assurance — confirming the spectra look like real EEG (1/f shape, no abnormal peaks, no flat channels).
2. **Sub-segmentation** of each 25 s trial into shorter overlapping windows.
3. **Per-subject z-score normalization** of every segment.

4.2 Sub-segmentation

Each of the 480 trials was split into **5 non-overlapping 5-second segments**, yielding $480 \times 5 = 2400$ **segments**, each of shape (32 channels, 640 samples). The choice of 5-second windows balances three considerations:

- Long enough to contain multiple cycles of the slowest band of interest (delta, 0.5–4 Hz → up to 12 cycles per window).
- Short enough to give the classifier multiple examples per trial.
- Compatible with the stationarity assumption — EEG within a 5-second window is reasonably stationary, much beyond that it isn't.

All 2400 segments inherited their parent trial's stress label.

4.3 Per-subject z-score normalization (critical step)

Each segment's values were normalized using statistics computed across **that subject's segments only**:

```
x_normalized = (x - mean_subj) / std_subj
```

where `mean_subj` and `std_subj` are computed across all 60 segments belonging to that subject (4 tasks × 3 trials × 5 segments). The result has zero mean and unit standard deviation within each subject.

This is the single most important preprocessing step for cross-subject generalization. EEG amplitudes vary 5–10× across subjects due to differences in skull conductivity, hair, electrode contact, and individual cortical sources. If a model trains on raw amplitudes, it will learn each training subject's amplitude scale as a feature and fail on a new person whose scale differs. Per-subject z-scoring strips out the between-subject scale and shifts, forcing the model to use *patterns* rather than *magnitudes*.

The trade-off is that any *informative* between-subject amplitude differences (e.g., people with higher resting alpha may experience stress differently) are also removed. This is an acceptable trade for cross-subject generalization. A within-subject classifier would do the opposite.

After normalization, segment statistics had mean ≈ 0 , std ≈ 1 globally, with reasonable bounds (typically $\pm 4 \sigma$).

4.4 Phase 2 outputs

Saved to `Results/phase2/` :

- `01_psd_comparison.png` — power spectral density plots for each task
- `02_beta_alpha_ratio.png` — preview of the canonical stress marker
- `03_segmentation_example.png` — visualization of the 5-second windowing
- `04_normalization_effect.png` — before/after z-score normalization
- `preprocessed_segments.npz` — the (2400, 32, 640) tensor of normalized segments
- `segment_labels.csv` — segment-level metadata (subject ID, task, trial, segment index, stress label)

5. Phase 3 — Feature Extraction

5.1 Rationale

For classical ML, hand-crafted features are essential — RFs and SVMs cannot consume raw multivariate time series effectively. The features chosen were a standard EEG feature bank, organized into three families:

Family	Features per channel	Total (32 channels)
Frequency-domain	6 (delta/theta/alpha/beta/gamma band powers + Beta/Alpha ratio)	192
Time-domain	7 (Hjorth activity, mobility, complexity; mean, std, skewness, kurtosis)	224
Nonlinear	2 (spectral entropy, Higuchi fractal dimension)	64
Total	15	480

5.2 Frequency-domain features (192)

Power in each of the five standard EEG bands per channel, computed via Welch's method:

- **Delta** (0.5–4 Hz) — drowsiness, deep sleep
- **Theta** (4–8 Hz) — emotional processing, memory
- **Alpha** (8–13 Hz) — relaxed wakefulness; suppressed under cognitive load
- **Beta** (13–30 Hz) — alert thinking, active cognition; elevated under stress
- **Gamma** (30–45 Hz) — high-level cognitive integration

The **Beta/Alpha ratio** is included separately as a sixth feature per channel because it is the canonical EEG

stress marker: a ratio > 1 generally indicates active cognition, and the change from baseline correlates with stress responses.

5.3 Time-domain features (224)

Per channel, seven values:

- **Hjorth activity** — total signal power (variance)
- **Hjorth mobility** — proportional to mean frequency
- **Hjorth complexity** — proportional to bandwidth
- **Mean, standard deviation, skewness, kurtosis** — distributional shape

Hjorth parameters are particularly useful for EEG because they have direct frequency-domain interpretations while being computed in the time domain — they're fast and stable.

5.4 Nonlinear features (64)

Per channel, two values:

- **Spectral entropy** — flatness of the power spectrum (high = white-noise-like, low = peaky)
- **Higuchi fractal dimension** — quantifies signal complexity in the time domain

Both are sensitive to changes in cognitive state and complement the linear features.

5.5 Top discriminative features

ANOVA F-test between the binary stress classes identified the most informative individual features. Unsurprisingly, **Alpha-band power in parietal and frontal channels dominated** — consistent with the alpha-suppression-under-stress literature. Beta/Alpha ratio in similar regions came next. The pattern is interpretable, which is one of the main arguments for hand-crafted features in EEG: every feature corresponds to something a neuroscientist can name.

5.6 Phase 3 outputs

Saved to `Results/phase3/`:

- `01_feature_distributions.png` — distributions per class for top features
 - `02_feature_correlation.png` — correlation heatmap (used to assess redundancy)
 - `03_top_features.png` — ranked F-statistic of top discriminative features
 - `features.npz` — the (2400, 480) feature matrix
 - `features_with_labels.csv` — features + metadata in tabular form
-

6. Phase 4 — Classical ML Classification

6.1 Leave-one-subject-out cross-validation

Phase 4 was the first phase to commit to subject-grouped evaluation. For each of the 40 subjects:

1. Hold out that subject's 60 segments as the test fold.
2. Train on the other 39 subjects' 2,340 segments.
3. Standardize features using statistics from the training fold only (no test-fold leakage).
4. Fit the classifier, predict on the test fold, record metrics.
5. Repeat for all 40 subjects. Aggregate.

This is the standard methodology. The implementation is in `Classification.ipynb`.

6.2 Classifiers

Three classical algorithms were evaluated, each with reasonable default hyperparameters:

- **SVM with RBF kernel** — strong baseline for moderate-dimensional feature problems
- **Random Forest** — ensemble baseline, captures nonlinear interactions
- **KNN (k=5)** — simple, interpretable baseline

6.3 Results

Binary task (Relaxed vs. Stress), LOSO-CV:

Classifier	Accuracy	Precision	Recall	F1	Time (s)
SVM (RBF)	0.579	0.598	0.658	0.627	47.1
Random Forest	0.587	0.607	0.657	0.631	39.0
KNN (k=5)	0.522	0.552	0.586	0.569	1.5

Three-class task (Relaxed / Low Stress / High Stress), LOSO-CV:

Classifier	Accuracy	Precision	Recall	F1	Time (s)
SVM (RBF)	0.475	0.414	0.475	0.437	43.6
Random Forest	0.474	0.456	0.474	0.434	41.9
KNN (k=5)	0.432	0.390	0.432	0.406	1.5

6.4 Interpretation

The binary RF result of **0.587** became the project's baseline reference number. Two observations:

- **The numbers are realistic for honest LOSO on a 40-subject dataset.** Comparing this against published 90%+ numbers on SAM 40 is comparing different evaluation regimes.
- **The three-class problem is much harder.** Accuracy at 0.47 is not far above chance (0.33 for three classes). The High Stress class is severely underrepresented, and the boundary between Low and High Stress in self-reports is noisy.

7. Phase 4B — Improved Classification

7.1 Goals

Phase 4 was a baseline. Phase 4B aimed to improve it through three orthogonal techniques:

1. **Within-fold feature selection** (avoiding leakage).
2. **Class balancing** for the imbalanced three-class problem.
3. **Hyperparameter tuning** via held-out validation.

7.2 Within-fold feature selection

A common pitfall in feature selection is to do it on the *full* dataset before cross-validation begins — which leaks information from the test fold into the selection. The correct approach is to perform feature selection *within each fold*, using only that fold's training data.

The implementation: for each LOSO fold, use ANOVA F-test on the training data to rank all 480 features, select the top 80, and train the classifier only on those. The test fold sees the same 80 features.

This selection found that consistently across folds, alpha-band powers in parietal and frontal channels topped the rankings, with Beta/Alpha ratios in the same regions following.

7.3 Class balancing

For the three-class problem, `class_weight='balanced'` was added to SVM and RF. This re-weights the loss function so that mistakes on the minority High Stress class are penalized more heavily, preventing the classifier

from defaulting to the majority class.

7.4 Hyperparameter tuning

A small held-out validation split was carved from the training fold (preserving the subject-group structure), and grid search was performed over each classifier's main hyperparameters (e.g., SVM `C` and `gamma`, RF `n_estimators` and `max_depth`). The best-tuned configuration was then applied to the full LOSO evaluation.

7.5 Results

Binary task, tuned + feature-selected:

Classifier	Accuracy	Precision	Recall	F1	Time (s)
SVM (tuned)	0.581	0.596	0.686	0.638	9.3
Random Forest (tuned)	0.581	0.592	0.708	0.645	25.6
KNN (tuned)	0.554	0.580	0.615	0.597	1.2

Three-class task, tuned + feature-selected:

Classifier	Accuracy	F1
SVM (tuned)	0.436	0.437
Random Forest (tuned)	0.443	0.424
KNN (tuned)	0.434	0.421

7.6 Interpretation

The Phase 4B improvements were modest:

- **Binary accuracy stayed essentially flat** (0.587 → 0.581 for RF). Feature selection didn't help because the unselected features apparently weren't actively hurting the ensemble.
- **Three-class accuracy improved very slightly** (0.474 → 0.443 RF — actually slightly worse with tuning, possibly due to fewer trees). The class balancing helped recall on minority classes but accuracy is already a poor metric here.
- **Training time dropped substantially** for SVM (47.1 → 9.3 s) — the dimensionality reduction from 480 → 80 features made the kernel computation much faster.

The binary RF result of **0.587** (from Phase 4) remained the project's reference point because it was the highest honest baseline observed. Phase 4B confirmed that further improvements via feature selection alone wouldn't easily push past 0.60.

8. Phase 5 — Deep Learning

8.1 Motivation

The thesis of deep learning for EEG is: instead of hand-crafting features (band powers, Hjorth parameters, fractal dimensions), let the network *learn* the right features from raw segments. If that hypothesis holds, a CNN given (32 channels × 640 samples) raw segments should match or beat a Random Forest given 480 engineered features.

8.2 How a CNN “sees” EEG

A 5-second EEG segment shaped (32, 640) is mathematically a 2D array — 32 “rows” of channels by 640 “columns” of time. For Conv2d input, an extra channel dim is added: (1, 32, 640) per sample, where the leading “1” is the image-channel dim (analogous to RGB but with a single channel) and the 32 is now the image's

“height”.

EEG has a structural asymmetry between its two axes:

- **The time axis carries frequency content.** How the signal oscillates.
- **The channel axis carries spatial content.** Which brain regions are active.

So we use two qualitatively different convolution kernels:

- **Temporal kernel of shape (1, K)** — slides along time, ignores channels. Each filter learns to detect a specific oscillation pattern. After training, these often look like band-pass filters.
- **Spatial kernel of shape (32, 1)** — combines all channels at one time point, ignores time. Each filter learns a spatial pattern — which weighted combination of electrodes is informative.

This temporal → spatial decomposition is the foundation of every successful EEG CNN architecture (Shallow ConvNet, Deep ConvNet, EEGNet). It maps almost one-to-one onto Phase 3’s hand-crafted features — temporal filters approximate the band-power computations, spatial filters generalize the channel selection that the Random Forest was doing implicitly.

8.3 Evaluation strategy: hybrid

Running full 40-fold LOSO-CV for every architectural experiment would have cost hours of CPU time per attempt — impractical for iterating. The chosen compromise was a **hybrid evaluation strategy**:

- **Phases 5.1-5.2:** rapid iteration using a single subject-grouped held-out split (32 train / 4 validation / 4 test). Same principle as LOSO (no subject leakage) but ~10× faster.
- **Phase 5.4:** full 40-fold LOSO-CV on the winning architecture for the final reported number.

This is a standard ML engineering pattern: iterate fast on a holdout, then rigorously validate the final choice.

8.4 Phase 5.0 — Pipeline setup

`Phase5_0_Setup.ipynb` does the plumbing every subsequent Phase 5 experiment depends on:

1. Loads `preprocessed_segments.npz` and `segment_labels.csv` from Phase 2.
2. Converts the three-class labels to binary (Relaxed = 0, anything else = 1).
3. Randomly partitions the 40 subjects (seeded for reproducibility) into 32/4/4 train/val/test groups.
4. Builds PyTorch `Dataset` and `DataLoader` objects.
5. Sanity-checks shapes, label distributions, and signal statistics.
6. Saves the split assignment to `subject_split.json` so all subsequent phases use the same partition.

Class balance per split came out:

Split	Relaxed	Stress	% Stress
Train (32 subjects)	845	1075	56.0%
Validation (4 subjects)	120	120	50.0%
Test (4 subjects)	145	95	39.6%

The training set being 56% Stress while the test set is 39.6% Stress turned out to be consequential — see Phase 5.1.

8.5 Phase 5.1 — Simple Custom CNN

8.5.1 Architecture

A small custom CNN built explicitly around the temporal → spatial → temporal → classify pattern:

```

Input (B, 1, 32, 640)
↓
Block 1: Conv2d(1 → 8, kernel=(1, 25)) → BatchNorm
        ↓ (B, 8, 32, 640) — eight temporal filters learned per EEG channel

Block 2: Conv2d(8 → 16, kernel=(32, 1)) → BatchNorm → ELU
        AvgPool((1, 4)) → Dropout(0.5)
        ↓ (B, 16, 1, 160) — all 32 channels mixed into 16 virtual electrodes

Block 3: Conv2d(16 → 16, kernel=(1, 13)) → BatchNorm → ELU
        AvgPool((1, 8)) → Dropout(0.5)
        ↓ (B, 16, 1, 20) — second-order temporal abstraction

Classifier: Flatten → Linear(320 → 2)
↓
Output (B, 2) — logits for [Relaxed, Stress]

```

Total parameters: **8,346**.

Three design points worth noting:

- **No activation between Block 1 and Block 2.** Lets the spatial conv operate on linear (BN-normalized) temporal features. This matches EEGNet and Shallow ConvNet — applying a nonlinearity before the channel mixing makes the spatial filters work on already-nonlinear quantities, which empirically hurts EEG performance.
- **bias=False in convolutions followed by BatchNorm.** BN's shift parameter makes the conv bias redundant; setting **bias=False** saves a small number of parameters.
- **ELU activation** rather than ReLU. EEG signals are roughly symmetric around zero (post-normalization); ReLU would zero out half the signal. ELU passes negative values smoothly.

8.5.2 Training

- Optimizer: Adam with $\text{lr} = 1\text{e-}3$, $\text{weight_decay} = 1\text{e-}4$
- Loss: standard CrossEntropyLoss (no class weighting)
- Max epochs: 40
- Early stopping on validation loss with patience = 7

8.5.3 Result — failure mode

Training stopped at epoch 9. Test accuracy: **0.483**.

This was below the majority-class baseline (0.604) — meaning the model was actively worse than predicting “Relaxed” for every example. The confusion matrix told the story:

	Predicted	
	Relaxed	Stress
True Relaxed	57	88
True Stress	36	59

The model predicted “Stress” 147 times out of 240 (61.3% of test predictions), even though the actual Stress rate in the test set was only 39.6%. The 61.3% prediction rate is suspiciously close to the **training set’s 56.0% Stress prior**.

Diagnosis: the model learned the training-class prior, not actual discrimination. On a test set where the prior is inverted, this fails badly.

Additional problem: validation loss reached its minimum at epoch 2 ($\text{val_loss} = 0.6856$) and rose continuously afterward — the model had begun overfitting before it had learned anything generalizable.

8.6 Phase 5.1b — Class Weights and Regularization

Three changes were made:

1. **Inverse-frequency class weights** in the loss function:

```
w_c = N_total / (n_classes × N_c)
```

For the training set (845 Relaxed, 1075 Stress):

```
w_Relaxed = 1920 / (2 × 845) = 1.136
w_Stress = 1920 / (2 × 1075) = 0.893
```

The weighted CE loss penalizes mistakes on the minority Relaxed class more heavily, removing the model's incentive to default to "Stress".

2. **Smaller architecture.** Filter counts dropped: $n_{\text{temporal}} 8 \rightarrow 4$, $n_{\text{spatial}} 16 \rightarrow 8$. Total parameters dropped to **2,318**.
3. **Stronger regularization.** dropout $0.5 \rightarrow 0.6$, weight_decay $1e-4 \rightarrow 1e-3$, learning rate $1e-3 \rightarrow 5e-4$.

Result: Test accuracy **0.550**, F1-macro **0.538**. Prediction rate of Stress dropped to **0.446** (much closer to the test set's actual 0.396 — the model was now well-calibrated rather than biased toward the training prior).

This was a clear win over Phase 5.1 and became the leading DL configuration. Validation accuracy peaked at **0.6125** — slightly above Phase 4B's RF baseline of 0.587 on the balanced validation set.

Diagnostic note: During training, validation loss and validation accuracy diverged — val_loss bottomed out at epoch ~4 while val_acc kept climbing until epoch ~10. This happens with class-weighted loss because the loss penalizes minority-class mistakes more heavily, so the loss landscape doesn't track accuracy directly. Saving the checkpoint at lowest val_loss saved an undertrained model. The fix (used in Phase 5.2 onward) was to monitor val_acc directly via a configurable `monitor` parameter added to the training utility.

8.7 Phase 5.2 — EEGNet

The standard architecture for EEG deep learning since Lawhern et al. (2018).

8.7.1 Architecture: EEGNet-8,2

```
Input (B, 1, 32, 640)
↓
Block 1a: Conv2d(1 → F1=8, kernel=(1, 64))      [temporal]
         → BatchNorm
↓
Block 1b: DepthwiseConv2d(F1 → F1×D=16, kernel=(32, 1), groups=F1=8)
         → BatchNorm → ELU → AvgPool(1, 4) → Dropout(0.5)
↓ (B, 16, 1, 160)
Block 2: SeparableConv2d (depthwise temporal + pointwise mix)
         → BatchNorm → ELU → AvgPool(1, 8) → Dropout(0.5)
↓ (B, 16, 1, 20)
Classifier: Flatten → Linear(320 → 2)
```

Total parameters: **2,258**.

The two architectural innovations that distinguish EEGNet from the Simple CNN:

Depthwise convolution. A normal `Conv2d(F1, F2, kernel)` computes every output filter as a weighted sum of *all* input filters. With `groups=F1`, the input filters are split into F1 separate groups, each producing D outputs independently. No mixing across input filters. The parameter count drops from $F1 \times F2 \times K$ to $F2 \times K$.

Conceptually, this means each temporal filter (each "frequency band") gets its own dedicated D spatial filters. The alpha-band spatial pattern is not constrained to be a mixture of the beta-band spatial pattern — they're learned independently. This matches the neuroscience: the spatial topography of alpha differs from that of beta.

Separable convolution. A factorization that's equivalent in expressiveness to a full Conv2d but cheaper. It runs in two steps: (1) a depthwise temporal convolution that learns temporal patterns per channel independently, then (2) a 1×1 pointwise convolution that mixes those temporal features across channels via a learned linear combination. This is the standard "depthwise separable" pattern used in MobileNet and several other efficient architectures.

The temporal kernel length is **64 samples** — that's 500 ms at 128 Hz. Much wider than the Simple CNN's 25 samples (~195 ms). EEGNet's authors found this length captures the dominant alpha/beta cycles in one filter.

8.7.2 Training and result

Same training setup as Phase 5.1b (class-weighted loss, lr = 1e-3, patience = 15, monitor = val_acc, weight_decay = 1e-4).

Test accuracy: **0.517**, F1-macro **0.512**. **Below** the Simple CNN v2's 0.550.

The validation accuracy peaked at **0.567 on epoch 3** and declined steadily afterward. Validation loss climbed from 0.69 to 0.80 with violent oscillations. The model was overfitting almost immediately — saved best state was effectively a barely-trained model.

8.7.3 Diagnosis

The likely cause: the implementation omitted the **max-norm weight constraints** from the original EEGNet paper. Lawhern et al. apply two hard-edged regularizers:

- Depthwise conv weights: L2 norm per filter clipped to 1.0
- Classifier weights: L2 norm per output row clipped to 0.25

These constraints prevent individual filters from growing arbitrarily large during training — a kind of regularization that's distinct from weight decay (which adds a soft pull toward zero in the gradient). Without max-norm, a small EEGNet can let one spatial filter dominate by growing its weights large, memorize idiosyncrasies, and overfit fast. PyTorch has no built-in max-norm constraint (Keras does, which is why most EEGNet code online is Keras), making it easy to forget.

The training trajectory in Phase 5.2 looked exactly like what max-norm is designed to prevent.

8.8 Phase 5.2b — EEGNet with Max-Norm

Two changes:

1. **Max-norm constraints implemented in PyTorch** and applied after every `optimizer.step()`. The method computes the L2 norm of each filter (or each classifier-output row) and rescales weights whose norm exceeds the cap. Direction is preserved; only magnitude is reduced.

```
def apply_max_norm_constraints(self, dw_max=1.0, fc_max=0.25):
    with torch.no_grad():
        # depthwise spatial conv
        w = self.depthwise_conv.weight
        norms = w.norm(p=2, dim=(1, 2, 3), keepdim=True)
        scale = (dw_max / (norms + 1e-8)).clamp(max=1.0)
        w.mul_(scale)

        # classifier linear
        w = self.classifier.weight
        norms = w.norm(p=2, dim=1, keepdim=True)
        scale = (fc_max / (norms + 1e-8)).clamp(max=1.0)
        w.mul_(scale)
```

2. **Lower learning rate** (5e-4 → from 1e-3) for slower, more careful updates.

8.8.1 Result

Test accuracy: **0.525**, F1-macro **0.509**. Slight improvement over Phase 5.2 (0.517), still below Simple CNN v2 (0.550). Validation accuracy peaked at **0.554**, below Simple CNN v2's 0.6125.

The training curves showed that max-norm did exactly what it was supposed to: the val_loss explosion of Phase 5.2 (0.69 → 0.80) was replaced with a gentle rise (0.69 → 0.74). The runaway-weight failure mode was prevented. But validation accuracy still plateaued at ~0.55 — the architectural constraint, not the runaway weights, was the binding limit.

8.8.2 Why EEGNet underperformed

After giving EEGNet the full standard recipe (depthwise + separable + max-norm + class weights + careful LR + val-acc monitoring), it still lost to the simpler custom CNN. Plausible reasons:

- **EEGNet's inductive biases are calibrated for motor imagery and P300 paradigms** — tasks with very stereotyped per-subject signals where the spatial-temporal binding is consistent. Cognitive stress is more diffuse and varies more across subjects.

- **The depthwise constraint may be too restrictive.** Forcing each temporal filter to have its own independent spatial filters trades expressiveness for parameter efficiency. With only 32 distinct subjects providing training signal, the cross-frequency-band spatial mixing the Simple CNN allows may actually be useful.
- **Random initialization variance** could account for some of the gap. A more comprehensive evaluation would average across multiple seeds. With single-seed evaluation, a 0.025 gap (0.550 vs 0.525) could include noise.

In any case, the engineering decision was clear: commit further compute to LOSO evaluation of the winning architecture rather than additional EEGNet variants. The next planned phase (5.3, a CNN+GRU hybrid) was skipped for the same reason — adding more capacity hadn't been helping.

8.9 Phase 5.4 — Full LOSO

The Simple CNN v2 architecture from Phase 5.1b was selected for the final rigorous evaluation.

8.9.1 Methodology

40 folds. For each subject i :

1. Hold out subject i 's ~60 segments as the test fold.
2. Train on the other 39 subjects' ~2,340 segments.
3. Compute class weights from those 39 subjects' labels (per-fold, no leakage).
4. Initialize a fresh model (deterministic seed per fold).
5. Train for exactly 12 epochs — no early stopping.
6. Predict on subject i 's segments. Record accuracy, F1, F1-macro, confusion matrix.

The choice of 12 epochs was not arbitrary: Phase 5.1b found that validation accuracy peaked around epoch 10–11 with the same architecture and similar training-set size. Fixed-epoch training simplifies LOSO substantially — nested LOSO (proper early stopping on a subject-grouped val set within each training fold) would require $40 \times 39 = 1,560$ models and ~26 hours of CPU. Fixed-epoch LOSO is the standard simplification.

8.9.2 Aggregate result

40-fold mean accuracy: **0.5525 $\pm$ 0.118**

- Median: 0.575
- Min: 0.300, Max: 0.750
- F1 (positive class): 0.602
- F1-macro: 0.500
- 18 / 40 subjects exceeded the Phase 4B RF baseline of 0.587
- 14 / 40 subjects exceeded the majority baseline of 0.604
- 7 / 40 subjects below random chance (0.500)

Global confusion matrix (pooled across all 40 folds, 2400 predictions):

	Predicted	
	Relaxed	Stress
True Relaxed	413	697
True Stress	377	913

Stress recall: $913 / 1290 = 70.8\%$. Relaxed recall: $413 / 1110 = 37.2\%$. The class-weighted training created an asymmetry favoring stress recall over relaxed recall, which is appropriate for a stress *detection* application (false positives cost less than missed stress, depending on the use case).

Run time: 43.5 minutes on Intel i7-1255U. Thermal throttling on the thin-and-light chassis was the main bottleneck — individual folds ranged from 39 to 267 seconds depending on background load and CPU temperature.

9. Results and Discussion

9.1 Final comparison

Phase	Model	Eval method	Accuracy	F1-macro
4	SVM (RBF)	LOSO	0.579	0.627*
4	Random Forest	LOSO	0.587	0.631*
4	KNN	LOSO	0.522	0.569*
4B	SVM (tuned + selected)	LOSO	0.581	0.638*
4B	RF (tuned + selected)	LOSO	0.581	0.645*
5.1	Simple CNN	Held-out (4 subj)	0.483	0.483
5.1b	Simple CNN v2	Held-out (4 subj)	0.550	0.538
5.2	EEGNet	Held-out (4 subj)	0.517	0.512
5.2b	EEGNet + max-norm	Held-out (4 subj)	0.525	0.509
5.4	Simple CNN v2	LOSO (40-fold)	0.552	0.500

* Phase 4/4B F1 values are positive-class F1, not macro-averaged — comparison is approximate.

The headline numbers to compare apples-to-apples:

- **Best classical ML, LOSO:** RF at 0.587
- **Best deep learning, LOSO:** Simple CNN v2 at 0.552

9.2 Why traditional ML beat DL on this task

Three factors likely contributed:

Feature priors compensate for small data. Band powers, Hjorth parameters, spectral entropy, and Higuchi fractal dimension encode decades of EEG research about what's informative. The Random Forest had a 480-dimensional feature space where each dimension was already meaningful. The CNN had to learn equivalents from 1,920 training segments. With only 32 distinct subjects providing those examples, the CNN couldn't generalize as robustly. The CNN was learning the feature engineering *and* the classifier; the RF was just learning the classifier. With limited data, that's a meaningful difference.

EEG signal-to-noise is low. Cross-subject signal is dominated by between-subject variation that has nothing to do with task labels. Hand-crafted features (especially relative ones like Beta/Alpha ratio) are inherently robust to this kind of variation. Learned filters have to discover that robustness from the data itself.

EEGNet's architecture matches a different problem. EEGNet was developed for motor imagery and P300 paradigms — tasks with sharp, stereotyped temporal/spatial signatures that recur reliably within a subject. Cognitive stress is more like an ambient mood state than a discrete event; the spatial-temporal binding is looser. The depthwise constraint that locks each temporal filter to its own spatial filters may be over-restrictive when the signal is this diffuse.

This finding is consistent with the broader EEG-DL literature. Roy et al. (2019) and Craik et al. (2019) both note that for small EEG datasets (under a few hundred subjects), traditional ML frequently matches or beats end-to-end DL. The pattern flips when datasets get larger.

9.3 Per-subject variance

The LOSO mean (0.552) hides large per-subject variation. Some subjects' EEG is easy for the model — four subjects scored above 0.70. Others are nearly impossible — three scored below 0.40, with one as low as 0.30.

This variance has several possible sources:

- **Label noise:** some subjects' self-reported ratings may not align with their actual physiological stress response. A subject who rated all tasks similarly would have effectively random labels.

- **Signal quality:** electrode contact and movement artifacts vary across recording sessions.
- **Individual neurology:** the stress markers in EEG vary in strength across people. Some individuals show clear alpha suppression; others show subtler patterns.
- **Statistical noise:** 60 segments per subject is small. A subject's accuracy estimate has substantial sampling variance.

Reporting this variance is important. A model that averages 0.55 across 40 subjects with std 0.12 is qualitatively different from a model that gives 0.55 consistently on every subject. The first might be unusable for an individual user; the second is reliably mediocre. The first is what we have.

9.4 F1-macro vs accuracy

For imbalanced data — which the held-out test (40% Stress) and individual LOSO folds are — accuracy is misleading. A model that always predicts the majority class can outperform a real classifier on accuracy alone.

F1-macro averages the per-class F1, giving equal weight to both classes regardless of frequency. For the LOSO result:

- Random Forest F1 (positive class): ~0.631
- Simple CNN v2 F1 (positive class): 0.602
- Simple CNN v2 F1-macro: 0.500

A majority-baseline classifier on this dataset would have F1-macro of 0.376 (it gets one class perfect and one class at zero). The CNN's 0.500 is meaningfully above that, confirming that real classification is happening even when accuracy looks unimpressive.

9.5 The dishonesty problem in EEG literature

This project's central finding is meta-methodological: reported accuracies on SAM 40 in published work range up to 95%, and almost none of those numbers use subject-grouped evaluation. The discrepancy between honest LOSO (0.55–0.65) and inflated k-fold (0.85–0.95) results across the EEG DL literature has been noted by multiple reviewers — Roy et al. (2019) and Saeidi et al. (2021) both quantified the collapse. The contribution of this project, more than the accuracy itself, is to provide a clean, reproducible baseline on SAM 40 evaluated honestly.

10. Limitations

10.1 Small dataset

40 subjects is modest. Modern deep learning thrives on hundreds to thousands of distinct samples; 32 training subjects per fold (or 39 in LOSO) is on the low end for end-to-end methods. With 200+ subjects, the DL/ML comparison might invert.

10.2 Self-reported labels

Stress ratings are subjective and depend on each subject's reference frame and personality. A high-conscientious subject may rate the arithmetic task as "high stress" while a low-conscientious one might rate the same task as "mild discomfort". This noise floor limits any classifier's achievable accuracy regardless of architecture.

10.3 Hardware constraints

All experiments ran on a thin-and-light laptop CPU (Intel i7-1255U, no GPU). Architectures were constrained to small parameter counts (under 10K) for tractable training time. A larger ConvNet or transformer might perform differently on this task with more compute, though the small dataset would likely still favor smaller models.

10.4 Single split for held-out evaluation

Phases 5.1–5.2b used a single subject-grouped held-out test set (4 subjects). Individual results from this set can shift by $\pm 5\%$ depending on which subjects were drawn — a more robust comparison would average over

multiple random splits. The Phase 5.4 full LOSO mitigates this by averaging over all possible held-out subjects.

10.5 Binary task only for Phase 5

Three-class classification was dropped in Phase 5 due to severe class imbalance (High Stress at 14.6%). A larger or more balanced dataset would enable revisiting the three-class formulation.

10.6 Single deep learning seed per experiment

Each Phase 5 experiment was run with a fixed seed, not averaged across multiple seeds. The 0.025-point gap between Simple CNN v2 (0.550) and EEGNet + max-norm (0.525) on the held-out test could plausibly include seed-variance noise. The Phase 5.4 LOSO mean is more robust because it averages across many initializations implicitly.

11. Engineering Decisions

This section documents the major engineering trade-offs made during the project and the reasoning behind each.

11.1 Per-subject z-score normalization

Decision: Strip between-subject amplitude/offset variation via per-subject z-scoring during Phase 2.

Alternative: No normalization, or global-corpus normalization.

Reasoning: Between-subject amplitude variation is the largest source of between-condition signal in raw EEG. Without removing it, the model would learn each subject's amplitude as a feature and fail on unseen subjects. The cost is that any genuine *informative* amplitude differences are also removed — for cross-subject classification, this is the correct trade.

11.2 Subject-grouped evaluation from Phase 4 onwards

Decision: LOSO-CV for all classical ML; subject-grouped held-out + LOSO for DL.

Alternative: Standard k-fold or stratified CV.

Reasoning: Subject-mixed CV inflates accuracy by 20–35 points on EEG tasks because the model learns to identify subjects rather than classify the task. The whole project would be uninterpretable if this were not done.

11.3 Hybrid eval for Phase 5

Decision: Single held-out for architecture comparison (Phase 5.1–5.2), full LOSO for the winning model (Phase 5.4).

Alternative: LOSO from the start.

Reasoning: Full LOSO on each architectural experiment would have taken $\sim 45 \text{ minutes} \times 5 \text{ experiments} = \sim 4 \text{ hours}$ of training time. Even with that, the per-experiment number would still be a single estimate, not a confidence interval. The hybrid pattern — fast holdout iteration, then rigorous LOSO at the end — is a common ML engineering compromise.

11.4 Class weights in the loss

Decision: Use inverse-frequency CE loss in all Phase 5 experiments after Phase 5.1's failure.

Alternative: Use plain CE and accept that the model will learn the prior.

Reasoning: Phase 5.1 showed that without class weighting, the model defaults to predicting the training majority class. On a test set with a different class balance, this produces accuracy below the majority baseline. Class weighting forces the model to actually discriminate, even if absolute accuracy doesn't change much.

11.5 Skipping Phase 5.3 (CNN+GRU)

Decision: Skip the planned optional CNN+GRU hybrid.

Reasoning: Phase 5.1b → 5.2 → 5.2b showed that adding architectural capacity wasn't helping. Each variant had comparable or fewer parameters than the next, and none beat the simplest custom CNN. Adding a GRU on top would have added capacity in a different dimension (temporal recurrence), but the evidence pointed at the data, not the architecture, being the binding constraint. The compute budget was better spent on Phase 5.4's LOSO evaluation.

11.6 monitor='val_acc' for class-weighted training

Decision: Save the best model by validation accuracy rather than validation loss for Phase 5.2 onward.

Reasoning: With class-weighted loss, validation loss and validation accuracy can diverge — the loss penalizes minority-class mistakes heavily, so loss can rise even when accuracy improves. In Phase 5.1b, the lowest-val_loss epoch was epoch 4 (val_acc 0.575), while the highest val_acc epoch was 10 (val_acc 0.6125). Saving on val_loss would have systematically picked an inferior model.

11.7 Max-norm constraints from the original EEGNet recipe

Decision: Implement max-norm in PyTorch after Phase 5.2's runaway-overfitting result.

Reasoning: Most PyTorch EEGNet ports skip this — the original was Keras and PyTorch has no built-in. Phase 5.2's val_loss explosion (0.69 → 0.80) was the textbook failure mode max-norm prevents. Adding it was a faithfulness fix; without it, claiming we'd "tested EEGNet" would have been misleading.

11.8 Hardware-friendly model sizes

Decision: Constrain all DL models to under 10K parameters.

Reasoning: The available hardware (CPU only) made larger models too slow to iterate on. This is a real-world constraint, but it also matches the data — with under 2K training segments, larger models would have overfit anyway.

12. Future Work

Several directions could meaningfully extend this work:

Cross-dataset generalization. Train on SAM 40 and test on another EEG stress dataset (e.g., DEAP, MAHNOB-HCI). This is the next level of generalization stress test and would inform whether the patterns learned are dataset-specific or genuinely transfer.

Per-subject calibration. A small amount of subject-specific data could be used to fine-tune a globally-trained model. This is the standard hybrid approach in EEG BCI systems and could shrink the per-subject variance substantially.

Larger architectures with data augmentation. With more compute, larger Transformer-style architectures with augmentation strategies (e.g., temporal masking, channel dropout, mixup across subjects) might overcome the small-data ceiling. This would test the hypothesis that the DL/ML comparison flips with more compute and data manipulation.

Three-class classification with balanced sampling. SMOTE-style over-sampling of High Stress or focal loss could revisit the three-class problem.

Interpretability analysis. Visualizing the spatial filters learned by the Simple CNN's spatial conv layer would compare them to the topographies known from EEG stress research. Strong alignment would be confirmation; misalignment would be a useful diagnostic.

Frequency-decomposed inputs. Instead of feeding raw segments, feed band-filtered versions (delta, theta, alpha, beta, gamma) as separate input channels. This is a hybrid approach that combines feature engineering with end-to-end learning.

Multi-task learning. Jointly predict the task (Stroop / Arithmetic / Mirror / Relax) and the stress level. The task labels are noiseless and could provide useful auxiliary signal.

13. Conclusion

This project built a full ML pipeline for binary cognitive stress classification on the SAM 40 EEG dataset, evaluating both classical machine learning and deep learning approaches with rigorous subject-grouped cross-validation throughout.

The headline accuracy numbers — 0.587 for the best classical ML (Random Forest with band powers and Hjorth parameters) and 0.552 for the best deep learning (a small custom CNN, evaluated via 40-fold LOSO-CV) — are realistic for this dataset under honest evaluation, and substantially below the 85–95% accuracies reported in published work that uses subject-mixed cross-validation. The deep learning models, including the standard EEGNet baseline, did not outperform feature-engineered classical ML on this task. This is consistent with recent EEG-DL literature on small datasets.

The project’s primary contribution is methodological: a clean, reproducible baseline on SAM 40 evaluated rigorously, accompanied by a transparent account of the engineering decisions, diagnostics, and iteration history that produced the result. The codebase is structured for extension to other EEG datasets, classifiers, or tasks.

The honest finding is that EEG cognitive stress classification across unseen subjects is hard on a 40-subject dataset, and that more sophisticated architectures do not automatically improve over carefully-tuned simpler ones. Both findings are useful for anyone trying to build on this work.

References

1. Ghosh, R., Phadikar, S., Deb, N., Sinha, N., Das, P., & Ghaderpour, E. (2022). *SAM 40: Dataset of 40 subject EEG recordings to monitor the induced-stress while performing Stroop color-word test, arithmetic task, and mirror image recognition task*. Data in Brief, 40, 107772.
2. Lawhern, V. J., Solon, A. J., Waytowich, N. R., Gordon, S. M., Hung, C. P., & Lance, B. J. (2018). *EEGNet: A compact convolutional neural network for EEG-based brain-computer interfaces*. Journal of Neural Engineering, 15(5), 056013. [arXiv:1611.08024](https://arxiv.org/abs/1611.08024)
3. Roy, Y., Banville, H., Albuquerque, I., Gramfort, A., Falk, T. H., & Faubert, J. (2019). *Deep learning-based electroencephalography analysis: a systematic review*. Journal of Neural Engineering, 16(5), 051001.
4. Craik, A., He, Y., & Contreras-Vidal, J. L. (2019). *Deep learning for electroencephalogram (EEG) classification tasks: a review*. Journal of Neural Engineering, 16(3), 031001.
5. Schirrneister, R. T., Springenberg, J. T., Fiederer, L. D. J., Glasstetter, M., Eggensperger, K., Tangermann, M., Hutter, F., Burgard, W., & Ball, T. (2017). *Deep learning with convolutional neural networks for EEG decoding and visualization (Shallow and Deep ConvNet)*. Human Brain Mapping, 38(11), 5391–5420.
6. Hjorth, B. (1970). *EEG analysis based on time domain properties*. Electroencephalography and Clinical Neurophysiology, 29(3), 306–310.
7. Higuchi, T. (1988). *Approach to an irregular time series on the basis of the fractal theory*. Physica D: Nonlinear Phenomena, 31(2), 277–283.

Appendix A — Hyperparameter Summary

Phase 5 deep learning hyperparameters

Hyperparameter	Phase 5.1	Phase 5.1b	Phase 5.2	Phase 5.2b	Phase 5.4
Temporal filters (F1)	8	4	8	8	4
Spatial filters	16	8	16 (depthwise)	16 (depthwise)	8
Temporal kernel	25	25	64	64	25
Dropout	0.5	0.6	0.5	0.5	0.6
Learning rate	1e-3	5e-4	1e-3	5e-4	5e-4
Weight decay	1e-4	1e-3	1e-4	1e-4	1e-3
Class-weighted loss	No	Yes	Yes	Yes	Yes (per fold)
Monitor	val_loss	val_loss	val_acc	val_acc	N/A (fixed epochs)
Max-norm	No	No	No	dw=1.0, fc=0.25	No
Max epochs	40	60	80	100	12 (fixed)
Patience	7	10	15	20	N/A
Total parameters	8,346	2,318	2,258	2,258	2,318

Phase 4 / 4B hyperparameters

Classifier	Phase 4	Phase 4B (tuned)
SVM	RBF kernel, C=1	RBF, C and gamma grid-searched per fold
Random Forest	n_estimators=100, max_depth=None	n_estimators and max_depth grid-searched
KNN	k=5	k grid-searched
Feature selection	None	Top 80 via ANOVA F-test, within fold
Class weighting	None	balanced for 3-class

Appendix B — Per-Phase File Outputs

```

Results/
├── phase1/
│   ├── 01_raw_eeg_all_tasks.png
│   ├── 02_zoomed_relax_vs_arithmetic.png
│   ├── 03_stress_label_distribution.png
│   ├── 04_channel_layout.png
│   ├── 05_data_quality_dashboard.png
│   └── stress_labels.csv
├── phase2/
│   ├── 01_psd_comparison.png
│   ├── 02_beta_alpha_ratio.png
│   ├── 03_segmentation_example.png
│   ├── 04_normalization_effect.png
│   ├── preprocessed_segments.npz      # (2400, 32, 640)
│   └── segment_labels.csv
├── phase3/
│   ├── 01_feature_distributions.png
│   ├── 02_feature_correlation.png
│   ├── 03_top_features.png
│   ├── features.npz                  # (2400, 480)
│   └── features_with_labels.csv
├── phase4/
│   ├── 01_confusion_matrix_binary.png
│   ├── 01_confusion_matrix_3-class.png
│   ├── 02_per_subject_accuracy_binary.png
│   ├── 02_per_subject_accuracy_3-class.png
│   ├── 03_model_comparison.png
│   └── classification_results.csv
├── phase4b/
│   ├── 01_improvement_binary.png
│   ├── 01_improvement_3-class.png
│   ├── 02_confusion_matrix_binary.png
│   ├── 02_confusion_matrix_3-class.png
│   ├── 03_top_features_binary.png
│   ├── 03_top_features_3-class.png
│   ├── best_hyperparameters.txt
│   └── classification_results_improved.csv
├── phase5/
│   ├── class_balance_per_split.png
│   ├── subject_split.json
│   ├── phase5_1_simple_cnn/
│   │   ├── training_curves.png
│   │   ├── confusion_matrix.png
│   │   ├── results.json
│   │   └── simple_cnn_best.pt
│   ├── phase5_1b_simple_cnn_v2/
│   │   └── (same structure)
│   ├── phase5_2_eegnet/
│   │   └── (same structure)
│   ├── phase5_2b_eegnet_maxnorm/
│   │   └── (same structure)
│   └── phase5_4_loso/
│       ├── per_fold_results.csv
│       ├── per_subject_accuracy.png
│       ├── accuracy_distribution.png
│       ├── global_confusion_matrix.png
│       ├── final_comparison.png
│       └── results.json

```

Last updated: completion of Phase 5.4 LOSO evaluation.